

P2 Single-Step Debugger Manual

Observe and Control Your Running P2 Code

November 2026

Version 1.0.2

Manual Organization

A teaching guide to the P2 single-step debugger

Getting Started

- What Single-Step Debugging Is
- Turning On Debugging
- The Debugger Window
- Your First Session
- Commands and Controls

Working With the Debugger

- Breakpoints
- Observing State
- Working Sessions
- DEBUG Display Windows
- Tips and Troubleshooting
- Appendix: Feature by Version

Iron Sheep Productions, LLC

P2 Knowledge Base Project

Contents

Copyright and License **5**

Chapter 1: What Single-Step Debugging Is **6**
What you can observe . . . 6
What you can control . . . 7
When to reach for it . . . 7
A note for readers coming from the P1 . . . 7
Which tool shows you the debugger . . . 7
How this manual is organized . . . 8

Chapter 2: Turning On Debugging and Starting a Session **9**
Step 1: Compile with debugging enabled . . . 9
Step 2: Run it from the debug host . . . 9
Step 3: Reach a breakpoint . . . 9
What “with debugging enabled” costs . . . 11

Chapter 3: The Debugger Window **12**
Pass 1 — Where you are, and how you step . . . 14
Pass 2 — The cog’s own state . . . 16
Pass 3 — What the cog reaches . . . 18
The tour . . . 20
A habit worth forming . . . 26

Chapter 4: Your First Session **27**

Chapter 5: Command Reference **29**
Keyboard commands . . . 29
Control-key combinations . . . 30
Mouse — clicks by region . . . 30
Mouse — the wheel . . . 33
Mouse — hover . . . 34

Chapter 6: Breakpoints **35**
The kinds of breakpoint . . . 35
Breaking on an event . . . 36
Setting an address breakpoint . . . 36
Conditional breakpoints in code . . . 37

Asynchronous break between cogs (COGBRK) 37

Chapter 7: Observing State **38**

Memory 38
Registers and the special-function registers 38
The watch list 41
The call stack 41
Smart pins and events 41
Reading the heat map 41

Chapter 8: Working Sessions **42**

PASM-level debugging 42
Multi-cog debugging 42
Finding memory corruption 43
Debugging interrupts 43
Timing and performance 44

Chapter 9: DEBUG Display Windows (Cross-Reference) **45**

Chapter 10: Tips and Troubleshooting **46**

Working effectively 46
Troubleshooting 46

Appendix A: Feature Availability by Version **47**

List of Figures

3.1 Pass 1 — where you are, and how you step. (Debugging cog 0.) 14
3.2 Pass 2 — the cog's own state. 16
3.3 Pass 3 — what the cog reaches: hub, pins, and interrupts. 18
3.4 The status strip: C, Z, PC, SKIPF, XBYTE, and CT. 20
3.5 The cog (REG) and LUT register heat-map columns. 21
3.6 The disassembly; the highlighted line is the next instruction (the current PC). . . . 22
3.7 The watch cluster: the Register Watch (tagged REG), the special registers, and the event flags. 23
3.8 The state band: execution mode, the call stack, interrupt status, and the pointers. 23
3.9 The cog-status indicators and the DIR / OUT / IN pin rows. 24
3.10 The Smart-Pin Watch strip, tagged RQPIN with a delta marker. 24
3.11 The hub-memory viewer: hex bytes, an ASCII column, and the activity heat map. . 25
3.12 The break-condition buttons and the Go control. 25

7.1 Special registers ($1F0–$1FF) 39

Copyright and License

Copyright © 2026 Iron Sheep Productions, LLC and Parallax Inc.

This work is licensed under the Creative Commons Attribution–ShareAlike 4.0 International License (CC BY-SA 4.0).

You are free to:

- **Share** — copy and redistribute the material in any medium or format
- **Adapt** — remix, transform, and build upon the material for any purpose, even commercially

Under the following terms:

- **Attribution** — You must give appropriate credit, provide a link to the license, and indicate if changes were made.
- **ShareAlike** — If you remix, transform, or build upon the material, you must distribute your contributions under the same license as the original.

To view the full license, visit: https://creativecommons.org/licenses/by-sa/4.0/

Trademarks

Parallax, Propeller, Spin, and the Parallax logo are trademarks of Parallax Inc. This license grants permissions under copyright only; it does not grant rights to use these trademarks, and adapted or redistributed copies must not imply endorsement by, or official status with, Iron Sheep Productions, LLC or Parallax Inc.

Chapter 1: What Single-Step Debugging Is

You have written a P2 program. You compile it, you run it, and it does not do what you expected. Maybe a value comes out wrong. Maybe one of your cogs seems to stop. Maybe everything *looks* fine but the timing is off. Now what?

Without a debugger, your only window into the running chip is whatever you remembered to print. You add a DEBUG line, run again, add another, run again — guessing where to look. It works, but it is slow, and you can only see what you thought to ask for in advance.

A **single-step debugger** changes the deal. Instead of watching from outside, you step *inside* the running program. You can stop it at an exact instruction, look at every register and memory location as it stands at that moment, run one instruction and watch what changed, and continue until the next point of interest. The chip pauses and waits for you.

This manual teaches you to use the P2's single-step debugger to do exactly that.

What you can observe

When the debugger has your program paused, you can see the live state of the cog you are looking at:

- **Cog and LUT memory** — all 512 longs of cog RAM and 512 longs of LUT RAM, with a *heat map* that shows which locations are being read and written.
- **The flags** — the C (carry) and Z (zero) flags.
- **The Program Counter (PC)** — exactly which instruction is next.
- **Registers** — any cog register, including the special-function registers (PTRA, PTRB, the I/O registers, the interrupt vectors, and more).
- **Hub memory** — the shared RAM, shown as hex and as text.
- **The call stack** — how deep into nested calls you are, and the return addresses.
- **Smart pins and events** — the configuration and state of the chip's peripherals.

> A *heat map* is a colored overlay that shows activity at a glance: locations being written show one color, locations being read another, quiet locations stay dark. You will meet it properly in Chapter 4.

What you can control

You are not just watching — you are driving:

- **Step** — execute exactly one instruction, then stop again.
- **Run** — let the program run freely.
- **Break** — stop a freely-running program right now.
- **Breakpoints** — mark places or conditions where the program should stop on its own: a specific address, a `DEBUG` statement, a cog starting up, an interrupt, or an event.

> A *breakpoint* is a marked spot where execution should pause so you can look around. The *watch list* is the debugger's running tally of the registers that changed most recently — it surfaces them for you automatically as you step, rather than you choosing them in advance.

When to reach for it

Use the single-step debugger when "add another print and run again" is not telling you enough — when you need to *see the machine's actual state* at a precise moment: tracking down a wrong value, understanding why a branch went the way it did, watching memory get corrupted, or following how two cogs interact.

A note for readers coming from the P1

If your background is the Propeller 1, two habits do not carry over, and this manual uses the P2 forms throughout:

- Start a cog with **`COGSPIN`** / **`COGINIT`**, not the P1's `cognew`.
- Read the system counter with **`GETCT`**, not the P1's `CNT`.

The single-step debugger itself is new to you regardless of which Propeller you came from — there was nothing quite like it on the P1.

Which tool shows you the debugger

One thing to settle before you start, because it decides how much of this manual applies to you.

The single-step debugger is a **P2 facility**: the chip cooperates by pausing your program and reporting its state. But the *window* you look at is drawn by the program on your desk, not by the P2 — so which tool you run matters. Three present it:

- **PNut** — the original. These windows were designed and implemented here first, and the other two follow it.
- **PNut-Term-TS** — the cross-platform host, the same windows on Windows, macOS, and Linux. **The screenshots in this manual were taken here.**
- **Spin Tools IDE** — a third implementation, developed alongside PNut-Term-TS.

The intent is that this manual covers all three. PNut and PNut-Term-TS present the same debugger, and what follows is written against both. Spin Tools IDE is being built to the same design and may well already match it — but *we* have not certified it against this text, and we would rather tell you that than let you assume it. We expect its developer to certify it in due course. Until then: if you are running Spin Tools IDE, take this manual as very likely accurate rather than as verified.

This manual describes how the debugger is *meant* to behave — the behavior PNut established and the others are built to. If a particular host lags that in some small way, the answer is a fix to that host, not a footnote here, so you will not find this text cataloguing which build does what.

How this manual is organized

Chapters 2–3 get you a running DEBUG session and oriented in the window. Chapter 4 walks you through your first session hands-on. Chapters 5–8 are the working reference: commands, breakpoints, observing state, and richer tasks. Chapter 9 points you to the DEBUG *display* windows (covered in their own manual). Chapter 10 is tips and troubleshooting, and the appendix tells you which P2 release first carried each capability.

Chapter 2: Turning On Debugging and Starting a Session

The debugger does not run all the time. You compile your program *with debugging enabled*, then run it from the debug host. This chapter gets you from source code to a paused program.

Step 1: Compile with debugging enabled

The P2 compiler is **`pnut-ts`**. Add the `-d` flag to build your program with debugging turned on:

RUN THIS

```
pnut-ts -d myprogram.spin2
```

Without `-d`, your `DEBUG` statements are stripped out and the debugger never appears — your program just runs normally. This is exactly what you want for a finished build, and it means you can leave `DEBUG` statements in your source.

Step 2: Run it from the debug host

You need a **debug host** — the program that downloads your binary and puts the debugger window on screen (see *Which tool shows you the debugger* in Chapter 1). This manual shows **`pnut-term-ts`**, which combines a serial terminal, the nine DEBUG display windows, a downloader, and the single-step debugger in one cross-platform application. Launch your compiled program through it to download to the P2 and open the debug session.

If you are running **PNut** instead, the same thing happens and the debugger window is the same; only the surrounding application differs. Read `pnut-term-ts` throughout as "your debug host."

Step 3: Reach a breakpoint

The debugger window appears the first time your program reaches a place that asks it to. There are three common ways that happens.

A DEBUG statement in Spin2

An *argument-less* `DEBUG` statement is your Spin2 breakpoint. A bare `DEBUG` (no parentheses) or an empty `DEBUG()` brings up the single-step debugger and pauses execution at that line:

```
PUB main() | a, b, sum
  a := 10
  b := 32
  DEBUG                             ' single-step debugger opens here
  sum := add_two(a, b)
  ' display output only — does NOT open the debugger
  DEBUG("sum = ", UDEC_(sum))

PRI add_two(x, y) : result
  result := x + y
```

A `DEBUG` that carries something inside the parentheses — a message or a value, like `DEBUG("sum = ", UDEC_(sum))` above — does **not** open the single-step debugger. That form sends formatted output to the DEBUG *display* windows instead (Chapter 9). To break into the single-step debugger, use the argument-less `DEBUG` or `DEBUG()`.

A DEBUG instruction in PASM

In assembly, the `DEBUG` instruction is your breakpoint. Execution pauses on it, and you single-step from there:

```
                ORG       0
start
                MOV       a, #10
                DEBUG                       ' execution pauses here
                ADD       a, #32
                DEBUG                       ' and here
                JMP       #start

a               RES       1
```

A cog starting up

When a cog is started with debugging active, the debugger can break as the cog begins — useful for catching a problem right at a cog's entry point. (You will turn this behavior on with the INIT breakpoint in Chapter 6.)

Opening the debugger automatically (CON-block settings)

The three ways above all place a breakpoint *in your code*. You can also make the debugger open automatically — without adding a single `DEBUG` line — by defining a constant in your program's top-level `CON` block. These are read at compile time, so you still build with `pnut-ts -d`. Two of them open the debugger, and a third narrows down which cogs are watched:

Constant	What it does
`DEBUG_MAIN`	Simply *defining* this symbol breaks at the very start of your program, ready to single-step from the first instruction.
`DEBUG_COGINIT`	Defining this symbol breaks every time a cog is started (COGINIT), catching each cog right at its entry point.
`DEBUG_COGS`	An 8-bit mask choosing *which* cogs have debugging enabled — bit 0 is cog 0 through bit 7 for cog 7. A cog whose bit is **clear** runs with debugging off entirely: its DEBUG statements produce nothing and the debugger will not break in it. Defaults to all eight.

`DEBUG_MAIN` and `DEBUG_COGINIT` are switches: their mere presence in the `CON` block turns the behavior on, so you write them with no value. `DEBUG_COGS` does take a value, because it carries the cog mask:

```
CON
  DEBUG_MAIN                  ' open the debugger at program start
  DEBUG_COGINIT              ' ...and again whenever a cog starts
  DEBUG_COGS    = %0000_0011  ' but only watch cogs 0 and 1
```

Reach for `DEBUG_MAIN` when you want to step from the very beginning; `DEBUG_COGINIT` is the one when the bug is in *how* a cog starts up. Use `DEBUG_COGS` to keep the noise down in a busy multi-cog program: clearing a cog's bit turns debugging off for that cog, so it never produces DEBUG output and the debugger never breaks in it — leaving you with only the cogs you actually care about. (`DEBUG_COGINIT` is the compile-time equivalent of the INIT breakpoint you will arm by hand in Chapter 6.)

What "with debugging enabled" costs

Debugging adds a small amount of code and RAM per cog so the chip can talk to the host. For a finished release you simply compile without `-d` and the overhead is gone.

Chapter 3: The Debugger Window

A Guided Tour

When the debugger opens, you are looking at the full live state of one cog at once, packed into a fixed character grid. It can feel busy — and the debugger does little to help at first glance, because **most of its regions are not labeled with what they are.** A few carry a terse tag (`REG`, `LUT`, `STACK`, `HUB`), a couple add only a small delta marker to a tag (the Register Watch and the Smart-Pin Watch), and several — the disassembly, the hint bar — carry no label at all. You are expected to recognize each region by *where it sits* and by *the text inside it*.

This chapter is the map that makes that easy — and it hands you the screen in **three passes** rather than all at once. Each pass shows the same screenshot with only one group of regions marked, and the table beneath it names those regions, tells you how to recognize each one, and — the part that matters most — tells you **how you interact with it right there.**

The three passes are:

1. **Where you are, and how you step** — the handful you use on every single step.
2. **The cog's own state** — the registers and execution state private to this cog.
3. **What the cog reaches** — hub memory, pins, and interrupts.

The numbering runs 1 to 19 across all three, and a region keeps its number wherever it appears, so "region 19" always means the same thing.

Each table's last two columns are the ones to come back to: **Mouse** and **Keys**, for *that region specifically*. The debugger has no menus, so what a region does is what you can click or press while you are over it — and where a Keys cell is empty, that region genuinely has no keyboard command of its own. Chapter 5 lists every command in one place; these columns tell you which of them belong to the thing you are looking at.

After the three passes the rest of the chapter is a zone-by-zone tour. You do not need to memorize any of it; just know that this is where to look when you are lost on the screen.

Pass 1 — Where you are, and how you step

These six are the working set: the ones you read and press on every step. Learn these and you can drive the debugger; everything else is detail you look up.



Figure 3.1. Pass 1 — where you are, and how you step.
(Debugging cog 0.)

#	Region	Find it on screen by...	What it shows	Mouse	Keys
1	Title bar	top edge: *Debugger - Cog N*	which cog this window is for	—	—
2	Status strip	top row: `C Z PC SKIPF XBYTE CT`	where you are: flags, PC, skip pattern, XBYTE state, system counter	click **PC** to re-lock the disassembly to the PC; hover **CT** for elapsed seconds	—
5	Disassembly	center; code lines, one highlighted	your code, decoded; the highlighted line is the next instruction	L-click = lock to PC · R-click a line = toggle an address breakpoint · wheel scrolls, faster with **Ctrl**, **Shift**, **Ctrl+Shift** (full step sizes in Chapter 5)	—
9	Execution mode	small tag below disassembly	`MAIN`, or `INT1/2/3` while in an interrupt	—	—
18	Break buttons & Go	bottom-right cluster around the big button	which break conditions are armed; run/step control	L-click a condition = set it exclusively · R-click = toggle it *and* clear DEBUG · L-click **Go** = run to next break · R-click **Go** = run through breaks	**SPACE** = Go · **ENTER** = run/stop · **B I D M** toggle BREAK/ INIT/DEBUG/ MAIN
19	Hint bar	very bottom edge (empty until you hover)	a one-line description of whatever you point at	hover any region to read what it is and how to use it	—

The hint bar is how you learn the rest of the screen. Hover the mouse over anything and region 19 tells you what it is and what a click will do, so you can read any region you have not met yet without leaving the window.

Pass 2 — The cog's own state

Everything in this group belongs to *this* cog and nothing else: its registers, its stack, its flags. On screen they wrap around the disassembly — the tall columns down the left, the register columns down the right.

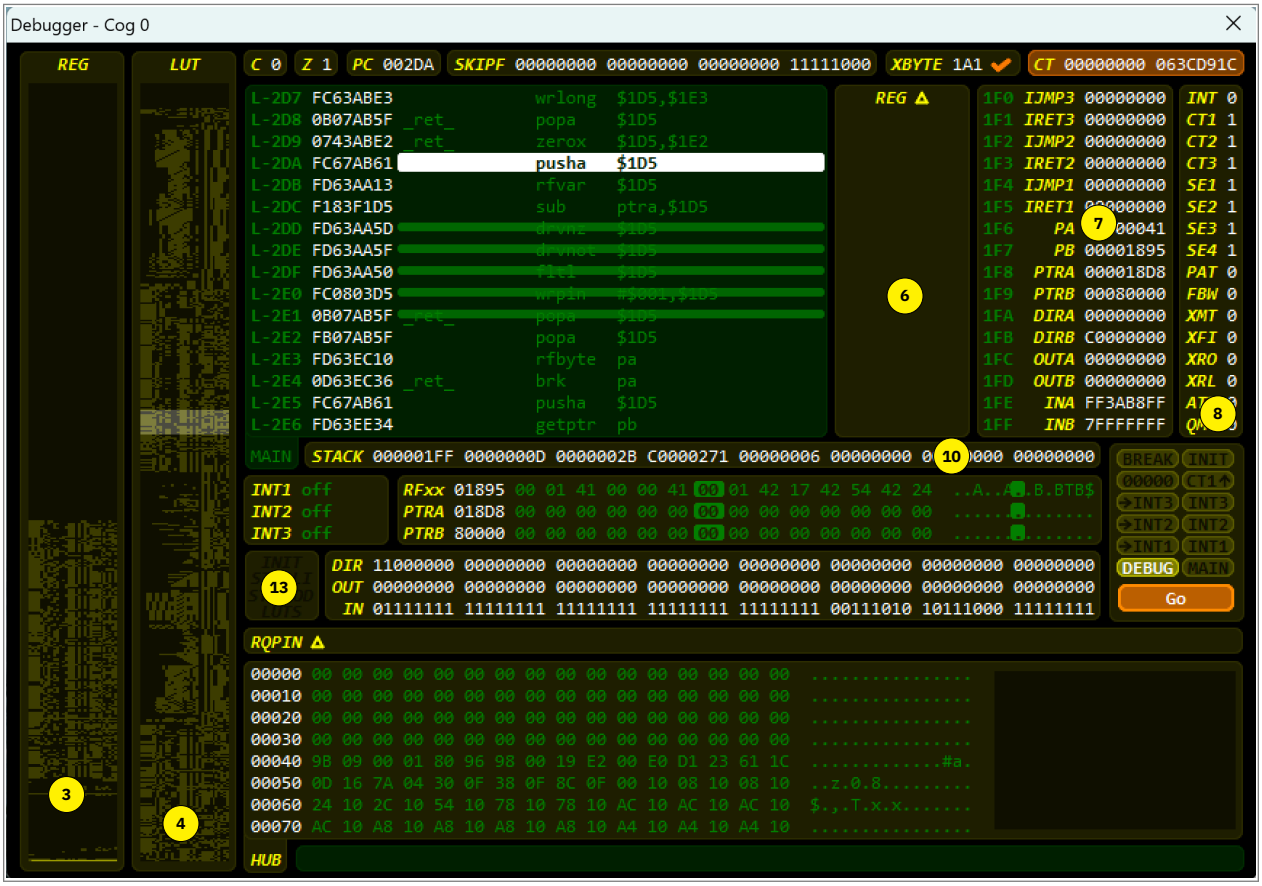


Figure 3.2. Pass 2 — the cog's own state.

#	Region	Find it on screen by...	What it shows	Mouse	Keys
3	Cog register map	far-left tall column tagged `REG`	heat map of all cog RAM ($000–$1FF)	click a spot to lock the disassembly to that cog address	—
4	LUT register map	2nd tall column tagged `LUT`	heat map of all LUT RAM ($200–$3FF)	click to lock the disassembly there	—
6	Register Watch	tagged `REG` with a delta marker, right of disassembly	cog registers that just changed	click the box to reset the list	**R** = reset this list
7	Special registers	register-name column, `IJMP3` through `INB`	the 16 special-function registers, $1F0–$1FF	click *any* value to follow it: `IJMP3`–`IRET1` reach cog space only when the value is below $400, and every other case is read as a hub pointer	—
8	Event flags	far-right column of event names (`INT`, `CT1`, … `QMT`), each `0/1`	which hardware events are set	L-click a name = arm a break on that event · R-click = toggle	—
10	Call stack	band tagged `STACK`, 8 hex values	the 8-level hardware CALL stack	click a value to jump the disassembly to that return address	—
13	Cog status	dim stack: `INIT STALLI STR MOD LUTS`	miscellaneous cog-state flags, lit when active	—	—

Pass 3 — What the cog reaches

The last group is everything beyond the cog itself: shared hub memory, the pins, and the interrupts that arrive from outside. These sit across the lower half of the window.

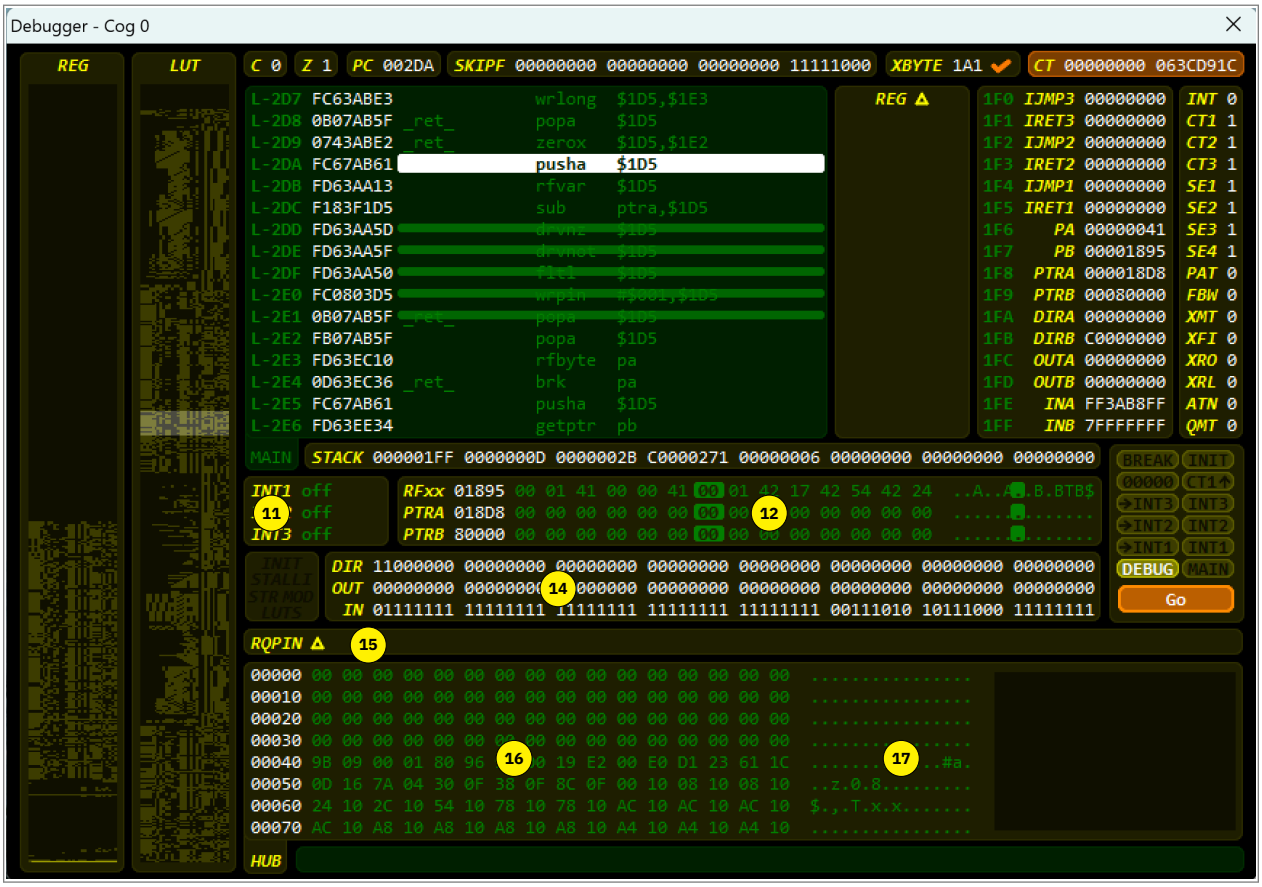


Figure 3.3. Pass 3 — what the cog reaches: hub, pins, and interrupts.

#	Region	Find it on screen by...	What it shows	Mouse	Keys
11	Interrupt status	left of the pointer band: `INT1/2/3`	each interrupt's state: `off / idle / wait / busy`	—	—
12	Pointers	rows `RFxx` / `PTRA` / `PTRB` + hub bytes	the FIFO and PTRA/PTRB, with the hub bytes around each	click **PTRA**/**PTRB** to jump the hub viewer there	—
14	Pin states	rows `DIR` / `OUT` / `IN`, 64 bits each	pin direction, output, and live input for all 64 pins	—	—
15	Smart-Pin Watch	one-row strip tagged `RQPIN` with a delta marker	smart pins whose `RQPIN` value changed	L-click = reset · R-click = reset **and** toggle the DIR-only/all-pins filter	—
16	Hub viewer	bottom band tagged `HUB`: address + hex + ASCII	shared hub RAM as hex and text	click a byte to jump there — hex and ASCII are separate regions, same effect · wheel over the address digits changes one nibble · wheel over the data scrolls it (step sizes in Chapter 5)	↑↓ = ±$10 · **PgUp**/**PgDn** = ±$80 (**Ctrl** $1000, **Shift** $10000)
17	Hub heat map	colored block right of the hub data	recent hub read/write activity	click a bright spot to jump the viewer to that 128-byte sub-block · the wheel does *nothing* here — click, do not scroll	—

The tour

The rest of the chapter walks the screen top-to-bottom, grouping regions the way your eye does. Each stop says what the region is *for* and folds in the ways you interact with it, so you learn "what this is" and "what I can do here" together.

The status strip (regions 1–2)

The **title bar** names the window — *Debugger - Cog N* — so you always know which cog you are looking at; when several cogs break, each opens its own window and you tell them apart here.

Just below it, the **status strip** is the handful of values that define *where you are*:

- **C** and **Z** — the carry and zero flags, as they stand after the highlighted instruction executes.
- **PC** — the Program Counter: the address of the next instruction. Click it to snap the disassembly back to the PC if you have scrolled away.
- **SKIPF** — the active skip pattern (or *Suspended during INTx* while an interrupt is running).
- **XBYTE** — the XBYTE bytecode-interpreter state.
- **CT** — the 64-bit system counter; it ticks constantly. Hover it and the hint bar reports the elapsed time in seconds.

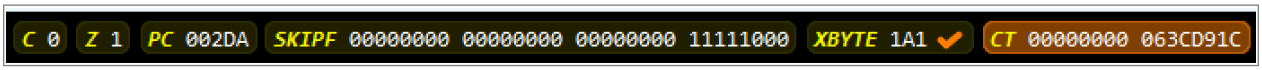


Figure 3.4. The status strip: C, Z, PC, SKIPF, XBYTE, and CT.

The register maps — REG and LUT (regions 3–4)

Two tall columns run the full height of the left edge: **REG** is all 512 longs of cog RAM ($000–$1FF), **LUT** is all 512 longs of LUT RAM ($200–$3FF). Both carry the *heat-map* overlay — brighter cells are being read or written, darker cells are quiet — so this is where you catch "something is hammering this location." Click anywhere in either column to lock the disassembly to that address and see the code living there.

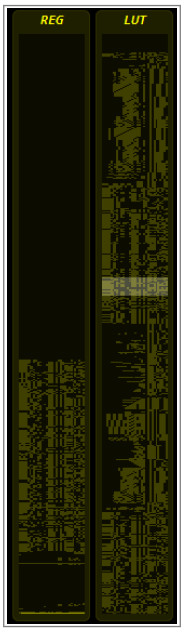


Figure 3.5. The cog (REG) and LUT register heat-map columns.

The disassembly (region 5)

The center of the window is your code, disassembled, with the **current PC line highlighted** (white background, green text) so you always see what runs next. This is where most of your attention lives. Three things happen right here:

- **Left-click** anywhere in the pane to lock it to *follow the PC* — handy after you have scrolled off looking around.
- **Right-click a line** to set or clear an **address breakpoint** there; a red marker appears at the left edge and the program stops when the PC reaches it.
- **Scroll** with the mouse wheel (hold **Ctrl** to move faster, **Shift** faster still). Scrolling detaches the pane from the PC until you left-click to re-lock.

You may see faint **strikethrough bands** across some upcoming lines: those are the instructions the live SKIP pattern will skip. They shift as the pattern changes — that is normal, not a rendering glitch.

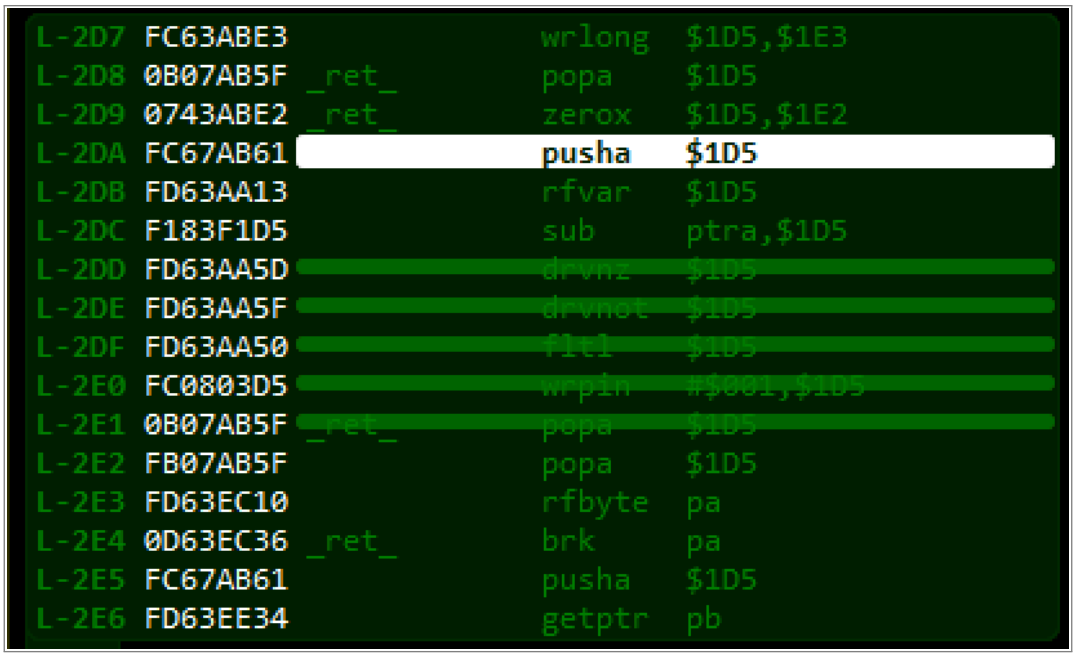


Figure 3.6. The disassembly; the highlighted line is the next instruction (the current PC).

The watch cluster — Register Watch, Special registers, Event flags (regions 6–8)

To the right of the disassembly sit three stacked columns that update as you step.

The **Register Watch** (tagged `REG` with a small delta marker) builds itself: as you step, it lists the cog registers whose values just changed, brightest for the freshest. You do not curate it — it surfaces exactly what your stepping is touching. Press **R**, or click the box, to clear it and start fresh. (Only ordinary registers $000–$1EF appear here; the special registers show in the next column instead.)

The **Special registers** column lists the 16 special-function registers, $1F0–$1FF, by name — the interrupt vectors, PA/PB, PTRA/PTRB, and the DIR/OUT/IN pin registers. Click a **PTRA** or **PTRB** value to jump the hub viewer straight to the address it holds. **INA/INB** are read-only — they mirror the live pins.

The **Event flags** column, at the far-right edge, lists the chip's hardware events (`INT`, `CT1`, `CT2`, … `QMT`), each with a `0` or `1`. Beyond just watching them, you can **click an event name to arm a break** on that event — the matching button over in the control cluster lights up to confirm.

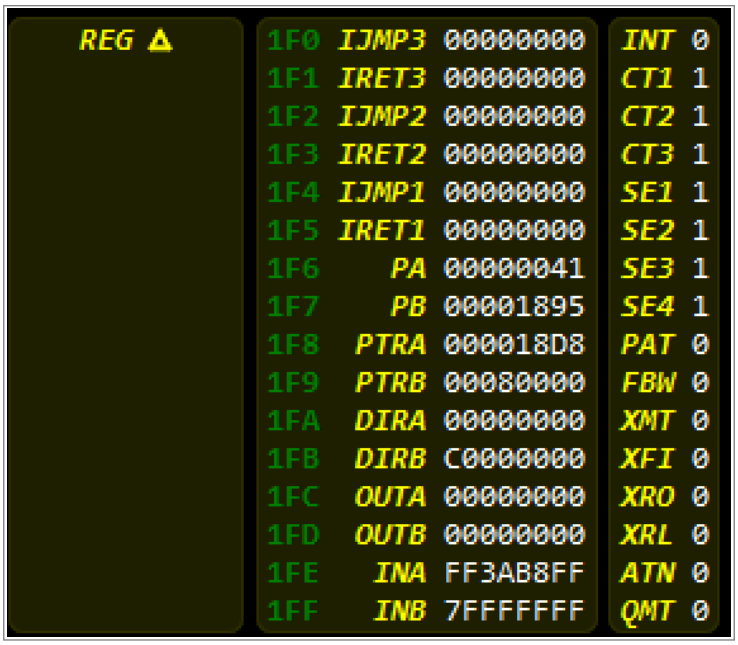


Figure 3.7. The watch cluster: the Register Watch (tagged REG), the special registers, and the event flags.

The state band — Execution mode, Call stack, Interrupt status, Pointers (regions 9–12)

A band across the middle reports how you got here and where your pointers aim.

- **Execution mode** reads `MAIN` normally, and flips to `INT1`/`INT2`/`INT3` while you are stopped inside an interrupt handler.
- The **Call stack** (`STACK`) shows the eight hardware CALL levels as hex return addresses. Click any value to jump the disassembly to that return address — the quickest way to see "who called me."
- **Interrupt status** shows `INT1/2/3` each as `off`, `idle`, `wait` (armed), or `busy` (its handler is running).
- **Pointers** shows the FIFO (`RFxx`) and `PTRA`/`PTRB`, each with a strip of the hub bytes around where it points. Click **PTRA** or **PTRB** to send the hub viewer there.

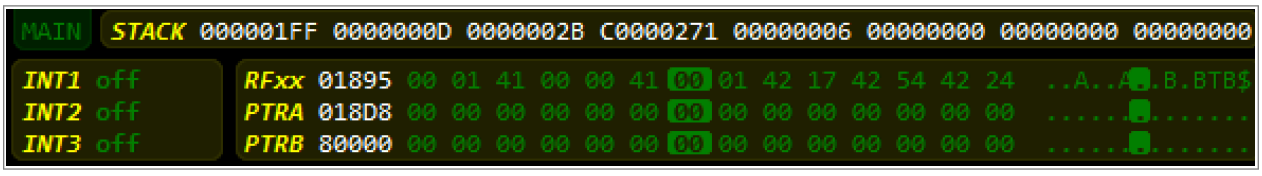


Figure 3.8. The state band: execution mode, the call stack, interrupt status, and the pointers.

The cog-status and pin band (regions 13–14)

Below the state band, a dim stack of indicators — `INIT`, `STALLI`, `STR`, `MOD`, `LUTS` — lights up when the corresponding cog-state condition is active and stays dark otherwise.

Beside it, the **pin states** show three rows — `DIR`, `OUT`, `IN` — of 64 binary digits each, split into two 32-bit halves. Bit 0 is the **rightmost** digit; the right-hand half is pins 31–0 (the *A* registers) and the left-hand half is pins 63–32 (the *B* registers). `DIR` is direction (1 = output), `OUT` is the driven level, and `IN` is what the pins actually read right now.

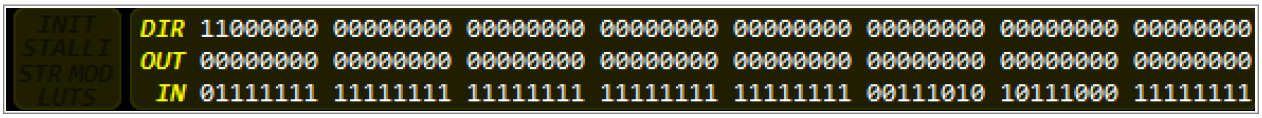


Figure 3.9. The cog-status indicators and the DIR / OUT / IN pin rows.

The Smart-Pin Watch (region 15)

A one-row strip tagged `RQPIN` with a delta marker, just under the pin rows, is the smart-pin counterpart to the register watch: it lists the smart pins whose `RQPIN` result changed as you step. **Left-click** it to reset the list; **right-click** resets it *and* toggles between showing only configured pins and all pins.



Figure 3.10. The Smart-Pin Watch strip, tagged RQPIN with a delta marker.

The hub viewer (regions 16–17)

The bottom band is the shared **hub** RAM: a 5-hex address, the hex bytes, and an ASCII column, with a **heat map** to its right showing recent activity. Move around with the arrow keys (one row, $10 bytes), PgUp/PgDn (a page, $80 — more with Ctrl/Shift), or the scroll wheel; jump straight to a location by clicking a byte, clicking a bright spot on the heat map, or clicking a pointer value elsewhere on screen. You can also dial an address in by scrolling the wheel over its individual hex digits.

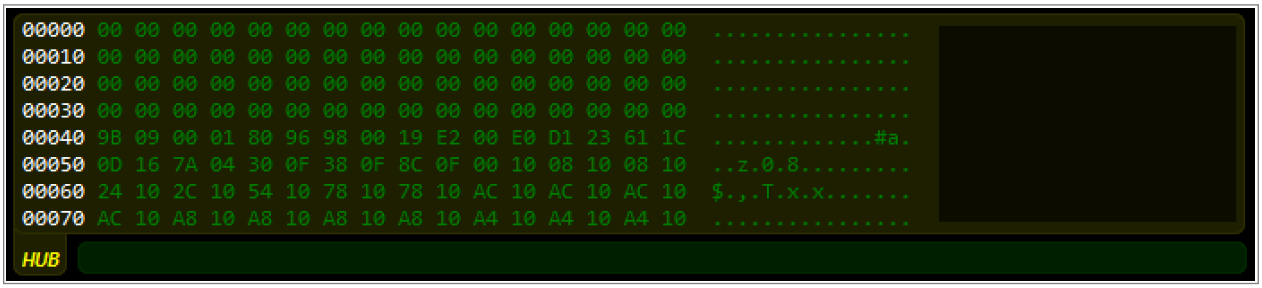


Figure 3.11. The hub-memory viewer: hex bytes, an ASCII column, and the activity heat map.

The break buttons and Go (region 18)

The bottom-right cluster is your control panel. The small buttons arm the break conditions (MAIN, the interrupts, DEBUG, INIT, an event, an address); **left-click** one to make it the only armed condition, **right-click** to toggle it alongside the others. The large **Go** button runs the program: press **SPACE** to go to the next armed break, or **ENTER** to run continuously through breaks (press ENTER again to stop). Chapters 5 and 6 cover these in full.



Figure 3.12. The break-condition buttons and the Go control.

A habit worth forming

Before you step, glance at three things: the **PC** (where am I), the **flags** (C/Z), and the **Register Watch** (what just changed). Step, then glance at the same three again. Most of a debugging session is that loop, with breakpoints (Chapter 6) to skip past the stretches you do not need to watch.

Chapter 4: Your First Session

Reading about the debugger only takes you so far. Let's drive it. This chapter uses the small program from Chapter 2; type it in (or use your own) and follow along.

```
PUB main() | a, b, sum
  a := 10
  b := 32
  ' argument-less: single-step debugger opens here
  DEBUG
  sum := add_two(a, b)
  ' has arguments: display output only, no break
  DEBUG("sum = ", UDEC_(sum))

PRI add_two(x, y) : result
  result := x + y
```

1. Compile with debugging and run it.

RUN THIS

```
pnut-ts -d firststep.spin2
```

Launch it from `pnut-term-ts`. The program runs until it reaches the argument-less `DEBUG`, then the single-step debugger window appears and execution pauses. (The second statement, `DEBUG("sum = ", UDEC_(sum))`, has arguments, so it only prints to the display windows — it will not stop here.) Your program is now waiting for you.

2. Get oriented. Find the **disassembly** pane and the highlighted current line — that is your PC. Glance at the **C** and **Z** flags in the status strip. Nothing has gone wrong; the chip is simply holding still.

3. Take one step. Press **Space**. Exactly one instruction executes, and the debugger stops again. Watch the highlighted line in the disassembly move, and watch the PC change in the status strip. You just executed a single instruction by hand. (Space really means "run to the next armed break"; because the default armed condition is MAIN, that works out to one instruction per press. You will meet the other break conditions in Chapter 6.)

4. Watch the values change. As you step toward `add_two`, keep an eye on the **Register Watch** pane: the debugger lists registers automatically as they change, so the result register shows up there on the step that writes it. Step through the addition and watch the value land — you are seeing the computation happen one instruction at a time.

5. Resume. Press **Enter** to let the program run again. It continues to the second `DEBUG`, prints `sum = 42`, and finishes.

That is a complete DEBUG session: **stop, look, step, look, resume.** The chapters that follow add breakpoints, so you can skip ahead to the part you care about instead of stepping from the start, and more places to look while you are stopped.

> Try this: run it again, and this time keep stepping *into* `add_two` instead of resuming. Watch the call stack pane gain a level as you enter the method and lose it as you return. That is the call stack tracking you will use to find your way through nested code.

Chapter 5: Command Reference

Chapter 3 introduced these controls where they live, region by region. This chapter is the **complete set**, gathered in one place to look up — every key, every click, every wheel step. Nothing here needs Chapter 3 to make sense, and nothing in Chapter 3 is left out. They are the same whether you reached the debugger from a Spin2 `DEBUG`, a PASM `DEBUG`, or a cog start.

Keyboard commands

Key	Action	What it does
Space	Go to next break	Run to the next armed break (same as left-clicking **Go**); with MAIN armed, that is one instruction per press
Enter	Run / stop	Run continuously through breaks, with the display updated at a throttled rate — about 20 breaks a second, not every one; press again to stop (same as right-clicking **Go**)
B	BREAK mode	Click the **BREAK** button — async-break mode; clears every condition **except INIT**
D	DEBUG toggle	Toggle break-on-DEBUG
I	INIT toggle	Toggle break-on-COGINIT
M	MAIN toggle	Toggle break-on-MAIN (single-step main code)
R	Reset watch	Clear the register-delta watch list. There is no LUT watch list — the LUT heat map is not a watch list and R does not touch it
↑ / ↓	Hub scroll	Scroll the hub data viewer one row (±$10)
PgUp / PgDn	Hub page	Page the hub data viewer ($80 per press; $1000 with Ctrl, $10000 with Shift)

Letter keys are case-insensitive, and they dispatch on the character you type rather than on the physical key position — so a non-QWERTY layout behaves exactly the same way.

> **No key switches between cogs.** Each cog that hits a breakpoint opens its **own window**, titled *Debugger - Cog N*; the windows cascade on screen as they open. To work on a different cog, switch to its window.

Some keys are captured and then deliberately do nothing. `Tab` is swallowed so keyboard focus cannot leave the debugger window, and **←**, **→**, **Home**, **End**, **Delete** and **Insert** are captured but reach no command. They are not missing features to work around — the window takes them and stops.

Alt and Cmd combinations are ignored entirely. Holding either does not fall through to the plain letter, so `Alt+M` is not `M`; it is nothing.

Control-key combinations

Five Ctrl combinations reach the hub-navigation commands instead of the letter commands. Every other Ctrl combination does nothing.

Key	Action
Ctrl+C	Hub scroll up one row (same as ↑)
Ctrl+D	Hub scroll down one row (same as ↓)
Ctrl+K	Hub page up, by `$1000` — the Ctrl-sized page, because Ctrl is genuinely held
Ctrl+L	Hub page down, by `$1000`, for the same reason
Ctrl+M	Run / stop (same as Enter)

CAUTION

Ctrl+D is not the DEBUG toggle. `D` on its own toggles break-on-`DEBUG`, but holding Ctrl reaches hub-scroll-down instead — a different command, in a different part of the window, with no visible complaint. If your DEBUG condition is not toggling, check whether Ctrl is down. The same applies to **Ctrl+M**, which is run/stop rather than the MAIN toggle: holding Ctrl *replaces* the letter command, it does not add to it.

One deliberate difference from PNut. `Ctrl+K` and `Ctrl+L` page by `$1000` here because the modifier state is read as it actually is at the moment you press them. PNut reads a state left over from a previous key, so the same combination can page by whatever tier happened to be set last. This is the straightforward reading rather than a reproduction of that behaviour, and it is noted so the difference is not mistaken for a fault.

Mouse — clicks by region

Left and right do different jobs in most regions, and in a few the difference is the whole point. The controls fall into two groups: the cluster that runs the program, and everything you click to look at something.

Running the program

Region	Left-click	Right-click
Break-condition button	set that condition **exclusively** — replaces the others, except INIT	toggle it on or off — **and clear DEBUG**. Four buttons depart from this; see Chapter 6
BREAK button	clear every condition except INIT	the same — BREAK does not distinguish the buttons
Go, cog halted at a break	run to the next break (same as Space)	run through breaks (same as Enter)
Stop, repeat mode running	stop	stop — either button
Break, cog free-running (display dimmed)	request an asynchronous COGBRK for *this* cog	the same

Go is a state machine, and it reads its state before it reads your button. That is why the left-/right distinction only means anything from a halted cog: while a repeat run is going, either button stops it, and while the cog is free-running, either button requests a COGBRK. Space and Enter follow the same rule.

When the cog runs for 250 ms without hitting a breakpoint the **display dims** — every pixel halved — and the button's caption changes to **Break**. That dimming is the window telling you what you are looking at is no longer current. Any press of Go also **flashes** the button, inverting its colours for about a tenth of a second, so a press is visible even when it changes nothing else on screen.

Looking at something

Most of these move the disassembly or the hub viewer to an address you clicked. One of them can refuse: a right-click address breakpoint is **declined in hub mode when the line resolves below `$400`**. Nothing appears to happen, and nothing is wrong — that address is cog space, which an address breakpoint set from hub mode cannot name.

Only four regions read which button you pressed — the break buttons, the disassembly, the event names, and the smart-pin watch box. Everywhere else a right-click does **exactly what a left-click does**. It is not ignored and it is not a second command, so "same" in the table below means the region genuinely acts, not that nothing happens.

Region	Left-click	Right-click
Disassembly	lock the disassembly to follow the PC	toggle an address breakpoint on the clicked line
REG or LUT heat map	lock the disassembly to the clicked address, placed mid-window	same
PC box	lock the disassembly to follow the PC	same
Special-function register	follow the value: `IJMP3`–`IRET1` reach cog space only when the value is below `$400`; every other case is read as a hub pointer	same
Stack value	follow the value; a hub-range value takes the disassembly with it	same
Pointer address	move the hub viewer *and* the disassembly there	same
Register Watch box	reset the list	same
Smart-Pin Watch box	reset the list	reset it **and** switch between all pins and only pins with DIR set
Hub data, hex	move the hub address to the clicked byte	same
Hub data, ASCII	the same, one character per byte — a separate region from the hex	same
Hub heat map	jump the viewer to that 128-byte sub-block	same
Event name	set the break event to that event **and arm it**	toggle that event break off or on

The REG and LUT heat strips are inset inside their boxes, and only the strip itself is clickable — the labelled box around it is hover-only. The register you click is placed **mid-window** rather than at the top, and clamped so a full sixteen-line window always stays in range.

Right-clicking on macOS. A physical right-press, a two-finger trackpad tap and **`Ctrl`+left-click** all deliver a right-click. macOS reports one physical right-press as more than one event, so the whole gesture is latched — a single right-press toggles an address breakpoint exactly **once**, not twice.

Mouse — the wheel

Every wheel region moves in four tiers, selected by the modifier you hold. The tiers are not the same everywhere, so they are worth reading across:

Modifier	Disassembly, cog mode	Disassembly, hub mode	Hub data
none	1 register	4 bytes (one long)	16 bytes (one row)
Ctrl	4 registers	16 bytes	1 byte
Shift	16 registers	64 bytes	4 bytes
Ctrl+Shift	32 registers	128 bytes	128 bytes (one sub-block)

Note the hub-data column: unmodified is the *coarse* step there and **Ctrl** the fine one — the reverse of the disassembly, where the modifiers only ever make the step bigger.

Over the disassembly, if the display is following the PC, the first scroll switches it to cog or hub lock mode, seeded from the address on screen. Cog-mode scrolling **stops** at `$000` and `$3F0` — it does not wrap around. In hub mode the disassembly and the hub viewer share a single hub address, so scrolling one moves the other.

Over the hub address digits, each scroll step changes the hex digit under the pointer by one, so you can dial an address in place.

Over the hub heat map, nothing — the heat map is deliberately excluded from wheel scrolling. Click it to jump instead.

Cog mode clamps; hub mode wraps. Cog-mode scrolling stops at `$000` and `$3F0` so the window always stays full, but hub scrolling **wraps** at 20 bits (`$FFFFF`) rather than clamping — scroll far enough past the top or bottom of hub RAM and the address comes round again.

Everywhere else the wheel does nothing. Only the disassembly box and the hub panel handle it; over the buttons, the PC, the watch list, the SFRs, the stack, the events, the pointers, the smart-pin watch or the REG/LUT strips there is no wheel action at all.

On macOS, `Shift`+wheel arrives as a horizontal scroll rather than a vertical one. It is folded back into a single step, so `Shift`+wheel scrolls exactly as it does on the other platforms.

Mouse — hover

Point at a region and the hint bar at the bottom of the window names what you can do there. It is recomputed on every mouse-move.

Pointing at	The hint bar shows
a break button, or **Go**	what that condition breaks on; **Go**'s text changes with its state
the REG or LUT box	what the box is
the REG or LUT **heat strip**	what a click there will do — the strip and its box read differently
the disassembly	its three actions: lock to PC, toggle a break address, and what the wheel does
an event row	the event under the cursor
CT	elapsed seconds implied by the tick count at the current clock frequency
XBYTE	a decode of the mode word, naming the addressing mode and whether C and Z are affected

Some regions clear the bar on purpose. Hover the SFR values, the stack values, a pointer's address, data or characters, the hub data bytes or characters, or the smart-pin watch box, and the hint bar goes **empty** rather than keeping the last text. That is deliberate, and it matches PNut — a hint invented for these was removed as a deviation from it. An empty bar over a value is not a fault.

Move the pointer off the window and the bar does not go blank. It falls back to a standing idle line naming the clock the P2 reported:

```
Clock frequency is 200,000,000 Hz
```

and holds it until the pointer returns. That is also what the bar reads when the window first opens, before the mouse has been over it — so the first thing the bar ever tells you is the clock, not nothing. Before the first breakpoint the frequency is not yet known, and the bar stays empty rather than claiming `0 Hz`.

Chapter 6: Breakpoints

A breakpoint is how you tell the program "stop when you get here" so you do not have to single-step from the very beginning. The P2 debugger supports several kinds at once.

The kinds of breakpoint

A break-condition register controls which conditions are armed:

Condition	The button reads	Stops when…
MAIN	`MAIN`	main code executes
INT1 / INT2 / INT3	`INT1` `INT2` `INT3`	the corresponding interrupt fires
INT1 / 2 / 3 entry	**→INT1 →INT2 →INT3**	that interrupt is *entered* — catches the handler's first instruction
DEBUG	`DEBUG`	a DEBUG statement or debug instruction is reached
INIT	`INIT`	a cog starts (COGINIT)
EVENT	the **event's own name**, e.g. **CT1↑**	the selected event triggers
ADDR	the **address**, e.g. `00000`	execution reaches that address
COGBRK	— (no button)	another cog requests an asynchronous break

Two of those buttons are not labeled with the condition name used above.

There is no button that says "EVENT". It wears the name of whichever event is selected, plus an up-arrow — **CT1↑** — and it shows that name dimmed even before you arm anything. Likewise **there is no button that says "ADDR"**: it displays the chosen address, reading `00000` until you set one by right-clicking a disassembly line.

And there are two buttons per interrupt. The plain `INT1` breaks whenever INT1 is executing; the arrowed **→INT1** breaks only as INT1 is *entered*. They sit in different columns and do different things, so when this manual says "click INT1" it means the plain one.

You arm and disarm these with the condition buttons in the bottom-right cluster. **Left-click** a button to arm that condition **exclusively** — every other condition clears, INIT excepted. **Right-click** toggles the one you clicked — **and clears DEBUG with it.** Three of them also have keyboard toggles from Chapter 5 — **D** (DEBUG), **I** (INIT), and **M** (MAIN). An armed condition shows bright, a disarmed one dim — there is no numeric "break value" on screen, so the button brightness *is* your confirmation of what is armed.

That DEBUG side-effect is easy to trip over: right-click any condition while break-on-DEBUG is armed and DEBUG quietly goes out. If you want both, arm the other condition first and toggle DEBUG **last**, or use the **D** key.

Four buttons that do not follow that grammar

The departures are what make the panel usable, so they are worth knowing rather than discovering:

Button	How it differs
INIT	Independent in both directions. Left-click **adds** it without clearing anything else; right-click toggles it alone. No other button ever clears it
DEBUG	Right-click keeps INIT *and* DEBUG before toggling, so toggling DEBUG cannot clear DEBUG out from under itself. DEBUG is exclusive to everything but INIT
EVENT	Mutually exclusive with ADDR. Right-click clears it if it is armed; otherwise it clears ADDR and arms EVENT with the selected event number
ADDR	The mirror image — right-click clears it, or clears EVENT and arms ADDR with the address you set by right-clicking a disassembly line

There is one exception to that rule. **BREAK, at the top of the cluster, lights up when *nothing* is armed.** It is not a condition that just switched on — it is the debugger telling you there are no break conditions left, so nothing will stop the program. Disarm your last condition and you will see it come on.

Breaking on an event

There is no button labeled "EVENT." Instead you choose *which* event in the event-flags column (region 8, far-right edge): **click the event's name** — CT1, say — and it arms as the break event. Its button in the cluster then lights, showing the event name with an up-arrow (for example CT1↑), which is your confirmation the event break is armed. With an event armed, pressing **Space** runs the program freely until that event fires, then breaks once — so a single Space here can cover many instructions, unlike the one-instruction step you get with MAIN armed.

Setting an address breakpoint

To stop when execution reaches a specific instruction, **right-click that line in the disassembly pane**. A marker appears on the line, and the program pauses when the PC reaches that address; right-click the line again to clear it. If the line you want is not on screen, scroll the disassembly to it first (mouse wheel, or click a pointer/stack value that lands there).

Conditional breakpoints in code

You can make a breakpoint fire only when a condition holds by guarding an argument-less `DEBUG` with an ordinary `if`:

```
if value > 100
  DEBUG                 ' single-step debugger opens only when value > 100
```

The debugger appears only on the iterations where `value > 100` — far more useful than stopping every time when you are hunting an occasional case. Use the bare `DEBUG` here: a `DEBUG` with arguments inside (such as `DEBUG(UDEC(value))`) would send output to the display windows instead of breaking.

Asynchronous break between cogs (COGBRK)

One cog can break another. Enable **BREAK** mode (press **B**) in the cog you want to be interruptible; the break can then be fired across to it while you are stopped in another cog's debugger. This is how you stop a misbehaving worker cog that is not hitting a breakpoint of its own (Chapter 8).

You can see when a cog is a candidate for this. A cog running free and not hitting any break gradually **dims**, and its **Go** button changes to read **Break**: that is the debugger telling you "this cog is running loose — click to force an asynchronous break." One limitation to remember: an asynchronous break only lands while some *other* cog is already sitting in its own debugger.

Chapter 7: Observing State

The reason to pause is to look. This chapter covers what you can inspect and how.

Memory

Cog memory ($000 – $1FF). 512 longs of cog RAM — your registers and code. The heat map shows read/write activity, and the **Register Watch** pane lists locations automatically as they change.

LUT memory ($200 – $3FF). 512 longs of lookup-table RAM, shared between cog pairs (0–1, 2–3, 4–5, 6–7) and often used for fast data.

Hub memory. The shared RAM, shown as hex with an ASCII column. Navigate with the arrow keys (up/down one row, $10 bytes), PgUp/PgDn (by page), the scroll wheel, or by clicking the hub heat-map or a pointer value to jump straight to a location. You can also dial an address in by scrolling the wheel over its hex digits.

Registers and the special-function registers

The special-function registers occupy the top of cog memory. Grouped by function, they are the interrupt vectors, the pointer/parameter registers, and the I/O registers:

Address	Register	Function
$1F0	IJMP3	Interrupt 3 jump address
$1F1	IRET3	Interrupt 3 return address
$1F2	IJMP2	Interrupt 2 jump address
$1F3	IRET2	Interrupt 2 return address
$1F4	IJMP1	Interrupt 1 jump address
$1F5	IRET1	Interrupt 1 return address
$1F6	PA	CALLD ret / CALLPA / LOC
$1F7	PB	CALLD ret / CALLPB / LOC
$1F8	PTRA	Hub pointer A
$1F9	PTRB	Hub pointer B
$1FA	DIRA	Direction P0–P31
$1FB	DIRB	Direction P32–P63
$1FC	OUTA	Output P0–P31
$1FD	OUTB	Output P32–P63
$1FE	INA	Input P0–P31 (read-only)
$1FF	INB	Input P32–P63 (read-only)

Figure 7.1. Special registers ($1F0–$1FF)

The table below gives each register’s read/write access — the detail you act on at the debugger:

Register	Address	R/W	Purpose
IJMP3	$1F0	R/W	INT3 jump vector
IRET3	$1F1	R/W	INT3 return address
IJMP2	$1F2	R/W	INT2 jump vector
IRET2	$1F3	R/W	INT2 return address
IJMP1	$1F4	R/W	INT1 jump vector
IRET1	$1F5	R/W	INT1 return address
PA	$1F6	R/W	Port A scratch / call argument
PB	$1F7	R/W	Port B scratch / call argument
PTRA	$1F8	R/W	Pointer A
PTRB	$1F9	R/W	Pointer B
DIRA	$1FA	R/W	Direction register — pins 0–31
DIRB	$1FB	R/W	Direction register — pins 32–63
OUTA	$1FC	R/W	Output register — pins 0–31
OUTB	$1FD	R/W	Output register — pins 32–63
INA	$1FE	R	Input register — pins 0–31 (read-only)
INB	$1FF	R	Input register — pins 32–63 (read-only)

In the debugger these appear in the **SFR** group: you watch them update as you step; **INA/INB** are read-only (they mirror the live pins), and clicking a **PTRA**/**PTRB** value jumps the hub viewer to that address.

The P2's 64 I/O pins are split into two banks of 32, and each of these registers covers one bank: the **A** registers (DIRA/OUTA/INA) address pins **0–31**, and the **B** registers (DIRB/OUTB/INB) address pins **32–63**. Within a register, bit *N* is the pin at that position in its bank — so DIRA bit 0 is pin 0, while DIRB bit 0 is pin 32 and DIRB bit 31 is pin 63. In the direction registers a bit of 1 makes the pin an output and 0 makes it an input. So when you are watching a pin, pick the register for its bank and read the bit at the pin's position within that bank (pin 40, for example, is DIRB/OUTB/INB bit 8).

The watch list

The watch list builds itself: as you step, the debugger lists the registers whose values just changed (up to 16 at once), brightening the freshest and fading them as they go quiet. You do not curate it — it surfaces exactly the activity you are stepping through. Press **R** (or click the watch pane) to clear it and start fresh; a parallel list does the same for smart pins.

The watch tracks the ordinary cog registers, $000–$1EF. The 16 special-function registers above that ($1F0–$1FF — PA, PB, PTRA/PTRB, the interrupt vectors, the pin registers) never appear in the watch; you read those in the **SFR** column instead. So if you are watching for a change to, say, PTRA and the watch stays empty, that is expected — look to the SFR column for it.

The call stack

The stack pane shows up to eight levels of the hardware CALL stack with their return addresses, and the disassembly indicates CALL depth — together they tell you how you got to where you are.

Smart pins and events

You can view smart-pin configuration and state to confirm a peripheral is set up the way you intended, and watch the chip's event flags (interrupts, counter matches, streamer events, pattern matches, and so on) to see which events are firing.

Reading the heat map

The heat-map overlay turns raw access into a picture: brighter/warmer cells are being written or read heavily, darker cells are quiet. A location lighting up when you did not expect it is often your bug.

Chapter 8: Working Sessions

With the moves and the panes in hand, here are the tasks you will actually use the debugger for.

PASM-level debugging

Drop a `DEBUG` instruction wherever you want assembly execution to pause, then single-step and watch registers and flags change instruction by instruction:

```
                ORG       0
                MOV       a, b
                DEBUG                     ' pause and inspect
                ADD       a, #1
                DEBUG
a               RES       1
b               RES       1
```

Stepping this way shows each instruction's effect on the registers and flags before the next one runs.

Multi-cog debugging

Each cog is debugged independently, and **each cog that breaks gets its own window** — titled *Debugger - Cog N*. To look at a different cog, switch to its window (the windows cascade on screen as they open). Because the cogs run in parallel, you typically arm breakpoints in each cog of interest and use COGBRK (Chapter 6) to coordinate stopping them.

A cog's window appears only once that cog actually exists and breaks. In the example below, the download stops `main` at its first instruction — *before* the `COGSPIN` — so at first you see only the *Cog 0* window. Step or run `main` through the `COGSPIN`, and the blink cog launches, hits its own `DEBUG`, and its window opens then. Do not expect the second window before the cog that owns it has started.

```
VAR
  LONG stack[64]

PUB main()
  ' break in main; the debugger opens here
  DEBUG
  COGSPIN(NEWCOG, blink(56, 5_000_000), @stack[0])
  repeat
    waitms(1000)
```

continues on next page →

↪ continued from previous page

```
PRI blink(pin, half) | t
  t := GETCT()
  repeat
    ' break in the blink cog (opens its own window)
    DEBUG
    PINTOGGLE(pin)
    t += half
    waitct(t)
```

Note the P2 idioms: **`COGSPIN`** to start the cog and **`GETCT`** / `waitct` for timing. Both breakpoints are *argument-less* `DEBUG` statements — only that form opens the single-step debugger. The blink cog opens in **its own window** (*Debugger - Cog N*); switch to that window to step through it on its own while `main` keeps running.

Finding memory corruption

When a value is being clobbered and you do not know by whom, lean on the heat maps. The debugger has no data watchpoint, but it *colors* every read and write:

1. Keep the cog/LUT heat-map columns (or the hub heat-map) in view and step or run between breakpoints, watching for write activity on the affected location.
2. Watch the surrounding locations too — corruption often spills across neighbors.
3. Once you suspect *which* instruction is doing the writing, set an address breakpoint on it (right-click it in the disassembly) to stop there and catch the culprit in the act.

Debugging interrupts

1. Arm the relevant interrupt breakpoint by clicking the **INT1**, **INT2**, or **INT3** button (alongside MAIN, so you can step up to the interrupt). When it fires, the **Execution mode** indicator flips from `MAIN` to `INT1`/`INT2`/`INT3` — that is how you know you are now stopped *inside* the handler.
2. Check the interrupt vectors in the SFR column (IJMPx / IRETx), and the **Interrupt status** region to see each interrupt as `wait` (armed) or `busy` (its handler running).
3. Watch the event flags to confirm which event drove the interrupt.

While you are inside a handler, the **SKIPF** panel reads *Suspended during INTx* rather than a skip pattern — a reminder that any SKIP sequence is paused for the duration of the interrupt. It clears when you step out through the return.

Timing and performance

Use the **CT** system counter to measure how long a section takes: note CT, run the section, note CT again. Because single-stepping itself is slow, measure timing by running between breakpoints, not by stepping.

Do not measure *across* a `DEBUG` statement either. Every `DEBUG` hands the cog to the debug ISR, and the cog does not resume until that message has finished going out the serial link — far longer than the code you were trying to time. A CT span that contains a `DEBUG` measures the reporting, not the program. Read CT on both sides of the section, then print the difference afterward:

```
t1 := GETCT()
work_you_are_timing()
t2 := GETCT()
DEBUG("elapsed = ", UDEC_(t2 - t1))   ' outside the measured span
```

Chapter 9: DEBUG Display Windows (Cross-Reference)

Beyond the single-step debugger, the P2 can stream live data to a set of graphical **DEBUG display windows** — a serial terminal, a logic analyzer, an oscilloscope, an XY scope, a plot, an FFT spectrum view, a bitmap display, and a MIDI view — all hosted in `pnut-term-ts` alongside the debugger.

Those windows are a large topic with their own commands and workflows, and they are covered in depth in the **P2 Debug Window Manual**. This manual stays focused on single-step debugging; when you want real-time plotting and visualization, turn to that manual.

What is worth knowing here: the same `DEBUG` statement that can pause execution can also send formatted output. When you display a value, use the P2's output formatters — `UDEC`, `SDEC`, `UHEX`, `SHEX`, `UBIN` — each with an optional trailing underscore (`UDEC_`, `UHEX_`, …) that suppresses the automatic label when you have already written your own:

```
DEBUG("count = ", UDEC_(count))     ' prints: count = 42
DEBUG(UDEC(count))                  ' prints: count = 42  (auto label)
```

Chapter 10: Tips and Troubleshooting

Working effectively

- **Start small.** A single breakpoint and the step key will solve most problems before you need anything fancier.
- **Let the watch list work for you.** It auto-surfaces the registers that just changed — read it as a running summary of what your stepping is touching, and press **R** to clear it when you want a fresh slate.
- **Let the heat map point you.** Unexpected activity is a fast way to localize a bug you cannot otherwise see.
- **Use COGBRK for parallel bugs.** Coordinated stopping is the key to making sense of multi-cog behavior.

Troubleshooting

Symptom	Likely cause / fix
Debugger never appears	Did you compile with `pnut-ts -d`? Without `-d`, DEBUG is stripped.
A cog will not respond	It may be stalled; check whether it is waiting (e.g. on an event or `waitct`).
Lost in hub memory	Click the hub heat-map to jump, or dial the address in with the scroll wheel.
A breakpoint never hits	Re-check the address and that the matching condition (ADDR/DEBUG/INT...) is armed.
One **Space** advanced *two* instructions	The first was an AUGS/AUGD prefix (emitted for a *##* 32-bit immediate). The prefix and the instruction it augments are atomic — the debugger cannot break between them — so one Space steps past both. This is correct, not a stuck key.
The Register Watch stays empty for a special register	PA, PB, PTRA/PTRB, and the pin/vector registers ($1F0–$1FF) never show in the watch — read them in the SFR column (Chapter 7).

Appendix A: Feature Availability by Version

The single-step debugger and its capabilities arrived over several P2 toolchain releases. If a feature below is missing for you, your toolchain predates it. For current work, use the latest release; everything in this manual is available there.

Capability	Available since
PASM-level single-step debugging, breakpoints	v35u
Broad P2 hardware compatibility	v35v
Robust exception handling / stability	v35g
Automatic clock-frequency adaptation; flash-debug support	v36
Advanced multi-cog debugging	v36
Auto-triggering scope displays	v41
Complete feature set	v51 and later

> Note: this manual documents the debugger as delivered in the current environment — compiled with `pnut-ts` and hosted in `pnut-term-ts` — where the interaction model is carried forward unchanged from the original P2 single-step debugger.